



Research article

xRWA: A cross-chain framework for interoperability of real-world assets

Yihao Guo^a, Haoming Zhu^b, Minghui Xu^{b,*}, Xiuzhen Cheng^{b,1}, Bin Xiao^{a,*}^a Department of Computing, The Hong Kong Polytechnic University, Hong Kong, China^b School of Computer Science and Technology, Shandong University, Qingdao 266237, China

ARTICLE INFO

Article history:

Received 17 September 2025

Revised 25 November 2025

Accepted 6 December 2025

Available online 20 January 2026

Keywords:

Real-World Assets

Cross-chain interoperability

Decentralized identifiers

Authentication

ABSTRACT

Real-World Assets (RWAs) serve as a bridge between traditional financial instruments and decentralized infrastructures. By representing assets such as bonds, commodities, and real estate on blockchains, RWAs can extend the scope of decentralized finance. Industry forecasts further indicate rapid growth in tokenized RWAs after 2025, underscoring their potential role in the evolution of digital financial markets. However, in the current multi-chain environment, RWAs face challenges such as repeated authentication across multiple chains and inefficiencies arising from multi-step settlement protocols. To address these issues, we present a cross-chain framework for RWAs that emphasizes identity management, authentication, and cross-chain interaction. The framework integrates Decentralized Identifiers and Verifiable Credentials with customized attributes to support decentralized identification, and incorporates an authentication protocol based on Simplified Payment Verification to avoid redundant verification across chains. Furthermore, this paper adopts a cross-chain channel that supports efficient RWA settlements, and we refine its design so that the channel does not need to be closed immediately after each settlement, thereby reducing on-chain cost. We implement the framework and evaluate its performance via simulations, which confirm its feasibility and demonstrate improvements in efficiency for RWAs in cross-chain settings.

© 2025 The Author(s). Published by Elsevier B.V. on behalf of Shandong University.

This is an open access article under the CC BY-NC-ND license

(<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

1. Introduction

Real-World Assets (RWAs) refer to the tokenization of off-chain assets such as government bonds, commodities, and real estate, enabling them to be represented and transacted on blockchain systems [1]. They are attracting increasing attention in Web3 finance. As of June 2025, the total market capitalization of global stablecoins is approximately \$250 billion, while the market size of RWA tokenization grew from \$8.6 billion at the beginning of 2025 to \$23 billion in the first half of the year, representing a 260% increase. Looking ahead, Citibank estimates that tokenized RWAs may reach a scale of around \$4 trillion by 2030, indicating their potential role in the future development of digital financial markets [2].

Blockchain [3–5] constitutes the key infrastructure for the tokenization and circulation of RWAs, providing transparency

and immutability for asset issuance and transfer. In recent years, blockchain systems have expanded rapidly. By June 2025, CoinMarketCap reported more than ten thousand active cryptocurrency platforms [6], suggesting that RWAs can be deployed across an increasing number of heterogeneous blockchains. Although such distribution facilitates broader adoption [4,7,8], the confinement of RWAs to single blockchains limits their financial utility, as assets cannot be traded across different chains. This situation is analogous to traditional financial markets where banks are unable to conduct transactions across borders, thereby restricting the mobility and efficiency of capital. Fortunately, cross-chain technologies [9–11], regarded as a primary means of overcoming information silos between blockchains, may provide a feasible approach to enabling RWA interactions across heterogeneous systems. Nevertheless, existing research and practice on RWAs remain confined to single-chain contexts and have not systematically examined their cross-chain realization (shown in Fig. 1). To address this limitation, we propose a cross-chain RWA framework that aims to resolve the main challenges inherent in cross-chain processes.

Cross-Chain Redundant Authentication. Compared with single-chain deployment, cross-chain scenarios require an RWA to undergo repeated authentications during inter-chain transfers.

* Corresponding authors.

E-mail addresses: mhxu@sdu.edu.cn (M. Xu), b.xiao@polyu.edu.hk (B. Xiao).

¹ Given Xiuzhen Cheng's role as Editor-in-Chief, Minghui Xu's role as Editor of this journal, they had no involvement in the peer-review of this article and had no access to information regarding its peer review. Full responsibility for the editorial process for this article was delegated to Dr. Yongjuan Wang.

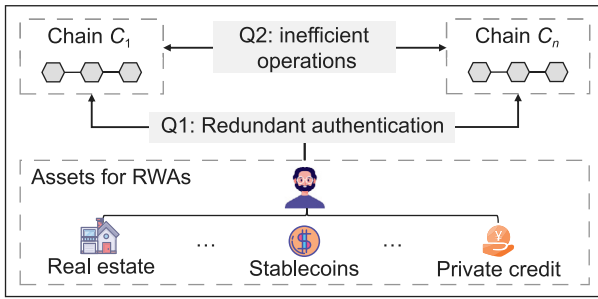


Fig. 1. The multi-chain environment creates opportunities for the cross-chain circulation of RWAs. However, redundant authentication and inefficient cross-chain operations remain critical challenges.

In a single-chain setting, an RWA only needs to be authenticated once within the local ledger, and its validity can then be consistently recognized across applications on that chain. In cross-chain environments, each blockchain maintains independent trust assumptions and verification rules. As a result, existing approaches typically require RWAs to undergo full authentication each time they are transferred on a different chain. Ideally, an RWA authenticated on one chain should be recognized by another through a proof of its prior authentication, thereby avoiding redundant procedures. Such a mechanism is particularly relevant when assets move from highly trusted chains, such as Ethereum mainnet, to less secure environments like its sidechains.

Inefficiency in Cross-Chain Operations. Even within a single blockchain, operations involving RWAs already incur latency due to consensus. For instance, the two most prominent blockchain platforms, Bitcoin and Ethereum, support approximately 7 TPS and 15 TPS, which is significantly lower than traditional payment networks such as Visa, that can process several thousand TPS [12]. In cross-chain environments, this inefficiency is further amplified, since completing a cross-chain interaction typically requires multiple interdependent operations across heterogeneous ledgers. To ensure atomicity, decentralized protocols such as Hashed Timelock Contracts (HTLCs) are commonly employed; however, they generally involve four distinct on-chain operations, which substantially prolong completion time and increase operational costs. Cross-chain channels [10,13] attempt to mitigate this limitation by integrating HTLC-based mechanisms with off-chain channels to improve efficiency. However, in existing designs, each settlement requires closing the channel and opening a new one for subsequent interactions, which introduces additional costs and delays. Therefore, the central challenge is to adopt cross-chain channels that support efficient interoperability for RWAs. Moreover, it is necessary to design mechanisms that avoid repeated channel closures during settlement, thereby enabling lower on-chain costs.

To fill this gap, we propose a cross-chain framework for RWAs to redefine the workflow of RWAs, with particular emphasis on identification, cross-chain authentication, and cross-chain interaction. To ensure decentralization throughout the process, we adopt Decentralized Identifiers (DIDs) and Verifiable Credentials (VCs), which are widely used in Web3, and extend them with customized attributes tailored to represent RWAs. Building on Simplified Payment Verification (SPV), we design a cross-chain authentication protocol that prevents redundant verifications of RWAs across multiple chains. Finally, we introduce a cross-chain channel scheme that enables the settlement of RWAs without requiring the channel to be closed after each settlement, thereby supporting efficient cross-chain interactions with lower costs.

For convenience, we highlight our main contributions as follows:

- (1) We propose a cross-chain framework for RWAs that considers authentication and efficiency in cross-chain processes. To the best of our knowledge, this is the first work in this direction.
- (2) To address the problem of redundant authentication across multiple chains, we design a cross-chain authentication protocol that enables RWAs authenticated on one chain to be recognized by others.
- (3) We employ a cross-chain channel to achieve more efficient RWA settlement. We further redesign the channel workflow to reduce operational overhead by eliminating the repeated opening and closing of channels.
- (4) We implement and conduct simulation experiments to demonstrate its feasibility and evaluate its performance.

The rest of the paper is organized as follows. In Section 2, we review the most related work. Section 3 introduces the necessary preliminary knowledge. In Section 4, we present our framework and discuss its applications. Section 6 reports simulation experiments that evaluate the performance of our scheme. Finally, Section 7 concludes the paper with remarks and insights.

2. Related works

In this section, we review related work on RWAs from both industry and academia.

2.1. RWAs in industry

RWAs have already been actively deployed in various industry sectors. MakerDAO [14] integrated tokenized U.S. Treasuries into its collateral framework, reinforcing the stability of decentralized finance. HSBC [15] introduced a gold-backed RWA token for retail investors in Hong Kong. Securitize [16] operates a regulated platform for issuing and trading RWAs, enabling compliant digital access to private companies. Brickken [17] has tokenized RWAs worth over USD 250 million across 14 countries. Goldman Sachs and BNY Mellon [18] partnered to tokenize shares of money market funds on blockchains, aiming to improve accessibility and settlement efficiency.

The RWA tokenization market has grown by more than 380% from 2022 to 2025 [19]. These deployments now span heterogeneous blockchain infrastructures such as Ethereum, Polygon, Solana, and permissioned networks like Canton, resulting in RWA assets being distributed across isolated blockchain environments without shared standards for interoperability or cross-chain recognition.

2.2. RWAs in academia

In academia, research on RWAs has primarily focused on single-chain architectures and survey studies. Chen et al. [1] analyzed the security of RWAs, highlighting risks in KYC/AML processes, oracle designs, and collateral structures. The work in [20] investigated the connection between RWAs and escrow mechanisms, and discussed potential directions for future development. Zhao et al. [21] proposed a scalable framework that integrates Ethereum staking, Layer-2 rollups, and DAO governance for RWA management. Xia et al. [22] studied tokenization approaches for RWAs with the aim of enhancing liquidity and improving asset management practices. LEGO [23] provided a quantitative methodology that maps historical failures in blockchain systems to current designs, including those involving RWAs.

In academia, current research on RWAs assumes a single underlying blockchain and lacks mechanisms for interoperability across heterogeneous chains. Existing studies do not address how RWAs deployed on different blockchain platforms can operate in multi-chain environments, resulting in fragmented deployments across multiple chains.

2.3. Summary and motivations

Recent developments indicate that RWAs have already been deployed across multiple sectors of the financial industry, with initiatives ranging from tokenized government bonds and money market funds to gold and real estate. Meanwhile, academic research has largely concentrated on single-chain architectures and survey studies. This contrast reveals a critical gap: although RWAs are deployed on different blockchain systems, they remain isolated within their respective environments. Each implementation therefore operates in a silo, limiting the broader financial utility of tokenized assets. Motivated by this observation, we propose a cross-chain framework for RWAs that addresses the challenges of redundant authentication and low efficiency, and enables secure interoperability across multiple blockchains.

3. Preliminaries

In this section, we present DIDs, VCs, simple payment verification, off-chain channels, and HTLCs, which serve as the building blocks of our work.

3.1. Decentralized identifiers

DIDs provide a decentralized identity mechanism for Web3, enabling users to prove ownership and control of their identities without relying on centralized authorities [24].

Definition 1 (DID Operations). The operations of a DID are represented as a tuple of polynomial-time algorithms $\text{DID} \stackrel{\text{def}}{=} (\text{KeyGen}, \text{Create}, \text{Resolve}, \text{Update}, \text{Deactivate})$:

- $(pk, sk) \leftarrow \text{KeyGen}(pp)$. Given public parameters pp , the algorithm generates a key pair (pk, sk) .
- $(did, doc) \leftarrow \text{Create}(pk, sk)$. Given a key pair (pk, sk) , the algorithm outputs a new identifier did and its associated document doc .
- $doc \leftarrow \text{Resolve}(did)$. Given an identifier did , the algorithm returns the current document doc linked to did .
- $doc' \leftarrow \text{Update}(did, doc)$. Given an identifier did and its document doc , the algorithm produces an updated document doc' , reflecting revised verification methods or service endpoints under the controller's authorization.
- $\emptyset \leftarrow \text{Deactivate}(did)$. Given an identifier did , the algorithm deactivates it, making the identifier invalid and non-resolvable.

DIDs have been widely adopted in Web3 ecosystems and are formally standardized by the World Wide Web Consortium (W3C) [25].

3.2. Verifiable credentials

VCS provide issuer-signed statements about a DID subject that can be verified by cryptographic proofs without trusting the presenter [26].

Definition 2 (VC Operations). Let the user be $\mathcal{U}$ with keys $(pk_{\mathcal{U}}, sk_{\mathcal{U}})$ and DID $did_{\mathcal{U}}$, the issuer be $\mathcal{I}$ with $(pk_{\mathcal{I}}, sk_{\mathcal{I}})$ and DID $did_{\mathcal{I}}$, and the verifier be $\mathcal{V}$. Let $\mathcal{R}^{\mathcal{I}}$ denote the issuer's revocation list. The process is represented by the algorithm tuple $\Pi \stackrel{\text{def}}{=} (\text{Req}, \text{Issue}, \text{Prove}, \text{Verify}, \text{Revoke})$:

- $req \leftarrow \text{Req}(\{d_i\}_{\mathcal{U}}, sk_{\mathcal{U}})$. Given data items $\{d_i\}$, $\mathcal{U}$ signs a request req to obtain a credential.
- $cred \leftarrow \text{Issue}(req, sk_{\mathcal{I}})$. Given req , $\mathcal{I}$ signs and returns a credential $cred$.

- $\pi \leftarrow \text{Prove}(cred, sk_{\mathcal{U}}, \phi)$. Given $cred$ and a predicate ϕ , $\mathcal{U}$ produces a proof π .
- $\{0, 1\} \leftarrow \text{Verify}(\mathcal{R}^{\mathcal{I}}, cred, \phi, \pi, did_{\mathcal{I}})$. $\mathcal{V}$ validates π , checks revocation against $\mathcal{R}^{\mathcal{I}}$, and confirms $did_{\mathcal{I}}$.
- $\mathcal{R}^{\mathcal{I}'} \leftarrow \text{Revoke}(\mathcal{R}^{\mathcal{I}}, cred, sk_{\mathcal{I}})$. $\mathcal{I}$ updates the status list to revoke or suspend $cred$.

An important feature of VCs is selective disclosure, which allows a holder to reveal only specific attributes of a credential while keeping all other information hidden [27].

3.3. Simple payment verification

SPV enables a verifier to verify whether a transaction is included in the blockchain without downloading full blocks [3].

Definition 3 (SPV). The SPV process is represented by a tuple of polynomial-time algorithms $\text{SPV} \stackrel{\text{def}}{=} (\text{Prove}, \text{Verify})$.

- $\pi \leftarrow \text{Prove}(tx, B)$. Given a transaction tx and a block B containing tx , the algorithm outputs a Merkle proof π , which consists of a Merkle path p authenticating tx with respect to the Merkle root μ in the header of B .
- $\{0, 1\} \leftarrow \text{Verify}(tx, \pi, \mu)$. Given a transaction tx , a Merkle proof $\pi = (p, \mu)$, and a Merkle root μ , the algorithm recomputes the authentication path using p and returns 1 if and only if tx is correctly included in the block, and 0 otherwise.

The Merkle path p denotes the sequence of sibling hashes along the path from the transaction node to the root of the Merkle tree, and μ is the Merkle root contained in the corresponding block header.

3.4. Off-chain channel

An off-chain channel allows two parties to perform frequent transactions without recording each interaction on the blockchain, thereby reducing on-chain overhead while maintaining security guarantees through cryptographic commitments [28].

Definition 4 (Off-Chain Channel). The off-chain channel process can be represented by a tuple of polynomial-time algorithms $\Pi \stackrel{\text{def}}{=} (\text{Open}, \text{Update}, \text{Close})$.

- **Open:** Both parties deposit assets into a contract on-chain, which locks funds as collateral for subsequent off-chain interactions.
- **Update:** The parties exchange signed messages off-chain to update balances or states. Each new state invalidates the previous one and is enforceable on-chain in case of disputes.
- **Close:** When the channel is terminated, the latest signed state is submitted to the contract, and the locked deposits are returned to the parties' accounts according to this final state, concluding the channel lifecycle.

Off-chain channels significantly improve scalability by shifting frequent interactions off-chain, while preserving security through cryptographic commitments and enforceability on-chain.

3.5. Hashed timelock contracts

HTLCs are conditional payment contracts widely used in cross-chain atomic swaps [10]. They ensure that either both parties complete the transaction successfully or both are refunded, preventing unilateral loss.

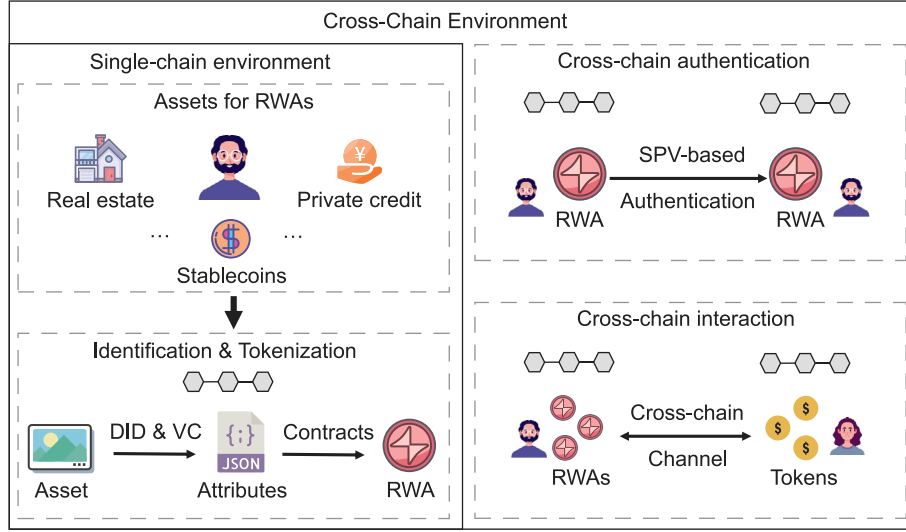


Fig. 2. xRWA comprises three functional layers: (i) asset identification and tokenization, where DIDs and VCs are used to represent RWAs on-chain; (ii) cross-chain authentication, which prevents repeated registration across heterogeneous blockchains; and (iii) cross-chain interaction through channel-based settlement, enabling efficient trading of RWAs.

Definition 5 (HTLC). HTLC consists of three polynomial-time algorithms $\text{HTLC} \stackrel{\text{def}}{=} (\text{Lock}, \text{Unlock}, \text{Refund})$. It operates on two blockchains $\mathbb{B}_0$ and $\mathbb{B}_1$, where both parties deploy contracts with the same hash value and coordinated timeouts.

- **Lock:** The initiator P_0 samples a random preimage ρ and computes its hash $h(\rho)$. P_0 locks an asset on $\mathbb{B}_0$ in a contract that specifies $h(\rho)$ and a timeout T_0 . The counterparty P_1 then deploys a corresponding contract on $\mathbb{B}_1$ using the same hash value and a shorter timeout $T_1 < T_0$.
- **Unlock:** To redeem the asset on $\mathbb{B}_1$, P_0 reveals ρ to the contract on $\mathbb{B}_1$ before T_1 . Since all blockchain data are public, P_1 learns ρ from this on-chain revelation and submits it to the contract on $\mathbb{B}_0$ before T_0 to unlock the asset. Both contracts verify that the submitted preimage satisfies $h(\rho)$ before releasing funds.
- **Refund:** If either party does not obtain ρ before the corresponding timeout, the contract on $\mathbb{B}_0$ (resp. $\mathbb{B}_1$) refunds the asset to P_0 (resp. P_1), ensuring no loss in the event of failed coordination.

HTLCs implement cross-chain payments by enforcing that assets are released only when the correct preimage is disclosed, and otherwise returned to the sender after the timeout, thereby ensuring atomicity.

In the following section, we adopt these techniques to design our proposed scheme.

4. xRWA

This section begins with an overview of xRWA with its design goals, followed by a detailed description of the proposed scheme.

4.1. Overview

xRWA provides a framework for RWAs that supports secure identification, cross-chain authentication, and efficient interoperability across heterogeneous blockchains. Fig. 2 shows the workflow, which consists of three layers: asset identification and tokenization, cross-chain authentication, and cross-chain interaction.

The process begins with the digital representation of off-chain assets such as real estate, stablecoins, and private credit assets.

Traditional approaches rely on centralized custodians or special-purpose vehicles that create contractual mappings between physical assets and their digital forms. xRWA adopts DIDs and VCs for decentralized identity management. By embedding customized attributes into VCs, the framework binds issuers and holders to asset records, ensuring legitimacy while reducing the reliance on intermediaries. After tokenization, assets may need to be recognized across multiple blockchains. Existing methods often require repeated verification on each chain. xRWA introduces a cross-chain authentication protocol based on SPV proofs. By linking DIDs and VCs with tokenized assets, and applying SPV validation, authentication performed on one chain can be reused across others. Cross-chain transfers also require efficient settlement. xRWA proposes a cross-chain channel that supports settlement without channel closures. This allows frequent interactions to occur off-chain, with only final states recorded on-chain, reducing on-chain costs.

4.2. Design goals

The security objective of xRWA is determined by two core guarantees that capture its correctness in multi-chain environments.

Cross-chain authentication correctness. A blockchain participating in xRWA may authenticate an RWA either (i) directly on its own ledger or (ii) by accepting an authentication result originating from another chain. For the second case, correctness requires that a chain C_t accept such a cross-chain authentication claim only if it corresponds to a genuine and confirmed authentication record on the source chain C_s according to the protocol rules. In particular, no PPT adversary should be able to fabricate cross-chain evidence so as to make C_t accept an RWA based on a non-existent or invalid authentication record.

Let $\text{AuthAcc}_{C_t}(a)$ denote the event that chain C_t accepts asset a via a cross-chain authentication claim from C_s , and let $\text{AuthRec}_{C_s}(a)$ denote the existence of a valid and confirmed authentication record for a on C_s . Authentication correctness requires that for every PPT adversary $\mathcal{A}$,

$$\Pr[\text{AuthAcc}_{C_t}(a) \wedge \neg \text{AuthRec}_{C_s}(a)] \leq \text{negl}(\lambda),$$

meaning that a cross-chain claim should never be accepted unless it is consistent with an actual authentication on the source chain.

Cross-chain settlement atomicity. When two blockchains engage in an RWA settlement procedure, the resulting state changes must be all or nothing. If settlement succeeds, both ledgers must apply the corresponding asset and payment updates. If settlement fails, both ledgers must retain their original states, and no honest party should end up with a partially executed outcome.

Let Com_1 and Com_2 denote the events that chains C_1 and C_2 , respectively, apply the settlement updates for a given cross-chain interaction. Atomicity requires that for every PPT adversary $\mathcal{A}$,

$$\Pr[\text{Com}_1 \oplus \text{Com}_2] \leq \text{negl}(\lambda),$$

where $\oplus$ denotes exclusive-or. Thus, it is infeasible for an adversary to cause exactly one chain to commit while the other does not, or to produce any partial or inconsistent settlement outcome.

Definition 6 (RWA Security). A cross-chain RWA protocol Π is said to satisfy *RWA security* if it ensures both (i) cross-chain authentication correctness and (ii) cross-chain settlement atomicity. Namely, for every PPT adversary $\mathcal{A}$, the probability that $\mathcal{A}$ causes a chain to accept a fabricated cross-chain authentication claim is negligible, and the probability that $\mathcal{A}$ produces an inconsistent or partially executed cross-chain settlement across two blockchains is negligible in the security parameter λ .

4.3. Identification and tokenization

To realize the identification of RWAs, we construct RWA-CC (RWA Composite Credential) using DIDs and VCs. The credential follows a modular design with four components: asset, identity, compliance, and custody. Each component includes its own *sStatus* and *sProof* to support independent revocation and verification, while a top-level proof guarantees overall integrity.

The asset component defines the core identification and tokenization of a RWA. The top-level *id* specifies the identifier of the credential instance itself. Within the asset object, the field *assetId* assigns a DID-based unique identifier to the asset. The field *assetType* indicates the broad category of the RWA, while *category* provides a finer classification within that type, for example distinguishing Residential from Commercial property. The *classDid* links the asset to a class definition maintained by an authoritative registry, enabling semantic grouping and interoperability across assets of the same class. A crucial element is *tBinding*, which establishes the binding between the off-chain asset record and its tokenized representation on a blockchain. It records the token standard, the target chain, the contract address, and the token identifier in a unified structure.

The identity component extends the description of an asset beyond its basic identifier. The field *schemaVersion* specifies the version of the identity schema, while *identitySchema* provides a reference URI that defines its structure. The *identifiers* array records registration information from official registries: each entry contains details such as the registration scheme, the identifier value, the jurisdiction, and the issuing authority. The following components may incorporate attributes specifically designed to uniquely identify various categories of RWAs, such as location, applicable standards, associated documentation, or other distinctive characteristics. These attributes record the intrinsic properties of the RWA and establish a one-to-one correspondence between the VC and the RWA entity.

The compliance component defines the regulatory framework governing an asset's circulation, including its authorizing license, permitted jurisdictions, applicable restrictions, validity period, and the regulator's identifier. Similarly, the custody component records safekeeping information: custodian identity, storage location, operational policy, audit frequency, and insurance reference.

After an RWA is identified through RWA-CC, the next step is to tokenize it on a blockchain. The process builds on the algorithms for DIDs and VCs introduced in Section 3. The issuer $\mathcal{I}$ generates its decentralized identity using *DID.KeyGen* and obtains a DID $did_{\mathcal{I}}$. The asset a is also assigned a DID did_a , which is included in the asset component of RWA-CC. The issuer then executes *VC.Issue* to generate a credential *cred* that encapsulates the attributes of a , including identity, compliance, and custody. This binds the off-chain asset to its verifiable description under the issuer's DID. To make the asset tradable, the credential is linked to an on-chain token via *tBinding*, which specifies the token standard (e.g., ERC-721), the blockchain, the contract address, and the identifier. This ensures that smart contracts interacting with the token can invoke *VC.Verify* against the issuer's DID and status list to check its validity. Finally, when a holder $\mathcal{U}$ presents the token in applications, only the necessary attributes are disclosed through *VC.Prove*, ensuring selective transparency while maintaining compliance guarantees.

```

1  "id": "<did:example:subject>",
2  "asset": {
3    "assetId": "<did:example:asset>",
4    "assetType": "<string>",
5    "category": "<string>",
6    "classDid": "<did:example:class>",
7    "tBinding": {
8      "standard": "<string>(e.g., 'ERC-721')",
9      "chain": "<string>(e.g., 'eip155:1')",
10     "contract": "<string>",
11     "tokenId": "<string>"
12   }
13 },
14 "identity": {
15   "schemaVersion": <number>,
16   "identitySchema": "<URL>",
17   "identifiers": [
18     {
19       "identifierScheme": "<string>",
20       "identifierValue": "<string>",
21       "jurisdiction": "<string>",
22       "issuingAuthority": "<string>"
23     }
24   ],
25   //Attributes applicable to specific scenarios
26   ...
27   "sStatus": {...},
28   "sProof": {...}
29 },
30 "compliance": {
31   "licenseId": "<string>",
32   "sellableRegions": [<string>],
33   "restrictions": [<string>],
34   "effectiveFrom": "<timestamp>",
35   "effectiveTo": "<timestamp>",
36   "regulatorDid": "<did:example:regulator>",
37   "sStatus": {...},
38   "sProof": {...}
39 },
40 "custody": {
41   "custodianDid": "<did:example:custody>",
42   "location": "<string>",
43   "policy": "<string>",
44   "auditCycleDays": <number>,
45   "insurancePolicyRef": "<string>",
46   "sStatus": {...},
47   "sProof": {...}
48 }

```

When an RWA is deployed to a single blockchain, the tokenization process described above ensures that its digital representation is securely bound to a DID and a VC. However, in

practice, RWAs often need to migrate across chains. A typical scenario resembles Web2 systems where an asset record, once canceled in one registry, must be reissued in another. Similarly, an RWA token may be burned on a chain C_1 and later reissued or transferred on C_2 to access new markets. Without interoperability, the tokenization procedure has to be repeated independently on each chain, leading to redundant authentication and increased operational overhead.

To overcome this limitation, we next introduce a cross-chain authentication protocol that allows an RWA verified on one chain to be recognized on another without repeating the full tokenization workflow.

4.4. Cross-chain authentication

Consider an RWA a that has been tokenized on C_1 , producing a credential cred anchored by a transaction tx in block B . To enable recognition of a on another chain C_2 without re-executing the full tokenization procedure, we employ SPV as defined in Section 3.3.

On C_1 , the issuer embeds a commitment to a within tx , which binds the asset identifier, the digest of the disclosed subset of cred , the token binding, and auxiliary parameters such as an epoch and nonce. The transaction tx becomes a leaf of the Merkle tree of B , with its membership authenticated by the Merkle root μ contained in the block header. To transfer authentication across chains, the issuer generates an SPV proof $\pi \leftarrow \text{SPV.Prove}(tx, B)$. The proof $\pi = (p, \mu)$ consists of the Merkle path p linking tx to the root and the Merkle root μ extracted from B . This compact proof, together with the serialized tx , is submitted to a verifier contract on C_2 . The verifier then executes $\text{SPV.Verify}(tx, \pi)$. It first checks that μ is consistent with a valid header of C_1 maintained under light-client rules. It then recomputes the path using p to confirm that tx is included under μ . If the check passes, the verifier parses the embedded commitment from tx and validates that it matches the disclosed fields of cred , including issuer DID, revocation status, and jurisdictional restrictions. If all conditions hold, the credential cred is accepted on C_2 as if it were locally issued. Through this mechanism, authentication completed on C_1 can be securely reused on C_2 .

Unlike conventional approaches where each blockchain must independently repeat the full verification of DIDs and VCs, the SPV-based protocol requires such verification only once on the source chain. Subsequent chains merely validate the compact SPV proof to ensure that the authenticated transaction has indeed been recorded on the source ledger and that the embedded commitment is consistent with the disclosed credential fields.

The detailed procedure of the SPV-based cross-chain authentication is summarized in Algorithm 1.

4.5. Efficient cross-chain interactions

In this part, we adopt HTLCs and off-chain channels to construct a cross-chain channel that supports partial settlement of RWAs without requiring channel closures. We first provide the definition of the cross-chain channel and then describe how it can be applied to the settlement of RWAs.

Cross-Chain Channel. A cross-chain channel [10] aims to guarantee atomicity while significantly reducing on-chain interactions. It allows parties to perform frequent cross-chain payments off-chain and only settle on-chain when necessary.

Definition 7 (Cross-Chain Channel). The process of this channel can be represented by a tuple of polynomial-time algorithms $\Pi \stackrel{\text{def}}{=} (\text{Open}, \text{Update}, \text{Lock}, \text{Unlock}, \text{Refund}, \text{Close})$.

- **Open:** Each party deposits assets into contracts on their respective chains, forming a bi-directional channel.

Algorithm 1 SPV-based cross-chain authentication

- 1: **Setup:** Asset a is tokenized on C_1 , producing a credential cred anchored by transaction tx in block B .
- 2: **Commitment (on C_1):** Issuer embeds into tx a commitment binding the asset identifier, digest of cred , token binding, and auxiliary parameters. tx is included in B , whose header contains the Merkle root μ .
- 3: **Proof generation (off-chain):** Compute $\pi \leftarrow \text{SPV.Prove}(tx, B)$, yielding $\pi = (p, \mu)$, where p is the Merkle path authenticating tx . Submit (tx, π) to C_2 .
- 4: **Verification (on C_2):** Verifier executes $\text{SPV.Verify}(tx, \pi)$. If valid, it parses the commitment in tx and checks consistency with disclosed fields of cred , including issuer DID, revocation status, and jurisdictional restrictions.
- 5: **Acceptance:** If all checks pass, cred is accepted on C_2 as if locally issued.

- **Update:** During the channel lifetime, the two parties exchange signed off-chain states to update balances. These updates do not require on-chain transactions, which improves efficiency.
- **Lock:** The parties submit the negotiated interaction results to the blockchain and lock their respective assets. Hash-locked conditions $h(\rho)$ are placed on both chains with timeouts $T_1 > T_2$ to guarantee atomicity.
- **Unlock:** If the receiver discloses the preimage ρ within the time limit T_2 , it can unlock the locked assets on one chain. Once ρ is revealed, the sender can use the same ρ to unlock the corresponding assets on the other chain.
- **Refund:** If the preimage ρ is not disclosed before the timeout, the locked asset is returned on-chain to the original depositor's account, and the channel state is updated accordingly.
- **Close:** When the channel is terminated, the remaining deposits locked in the channel are returned on-chain to the respective parties' accounts, thereby releasing the channel balance and concluding the channel lifecycle.

The above definition shows that settlement of RWAs can be achieved within the channel lifecycle through the Unlock and Refund procedures, without requiring channel closures. This feature allows partial settlements to be executed while keeping the channel open, thereby reducing the cost of repeated channel establishment and improving the efficiency of cross-chain interactions.

Cross-Chain Interactions. First, the parties invoke $\Pi.\text{Open}$ by deploying contracts ξ^1 and ξ^2 on C_1 and C_2 , which establish the channel environment and record deposits. During the lifetime of the channel, they repeatedly apply $\Pi.\text{Update}$ to exchange signed off-chain states describing tentative transfers of different subsets of RWAs. For example, a state σ_j may specify that B pays v_j units on C_1 in exchange for batch $S_j \subseteq \{a_1, \dots, a_m\}$. These updates remain entirely off-chain, enabling multiple negotiations without incurring blockchain overhead. Once the parties agree on a settlement, $\Pi.\text{Lock}$ is executed: both sides upload the agreed final state to ξ^1 and ξ^2 , where hash-locked conditions $h(\rho)$ with timeouts $T_1 > T_2$ are installed to ensure atomicity. If B reveals the preimage ρ on ξ^2 before T_2 , $\Pi.\text{Unlock}$ delivers the agreed union of RWAs $\bigcup_j S_j$ to B and allows S to redeem the corresponding aggregated payment on ξ^1 before T_1 . If instead B does not reveal ρ in time, $\Pi.\text{Refund}$ is triggered: ξ^2 returns the RWAs to S at T_2 , and ξ^1 returns the funds to B after T_1 . Finally, once the session concludes, $\Pi.\text{Close}$ is called to release any remaining deposits and terminate the channel.

Algorithm 2 Efficient cross-chain interactions

- 1: **Participants:** Buyer $\mathcal{B}$ (funds on C_1), Seller $\mathcal{S}$ (RWAs $\{a_1, \dots, a_m\}$ on C_2).
- 2: **Contracts:** ξ^1 on C_1 , ξ^2 on C_2 .
- 3: **Parameters:** deposits d_B, d_S , preimage ρ , hash $h(\cdot)$, timeouts $T_1 > T_2$.
- 4: **Channel opening (on-chain):**
 Π .Open: initialize ξ^1 and ξ^2 and record deposits ($\mathcal{B} \rightarrow \xi^1$: d_B , $\mathcal{S} \rightarrow \xi^2$: d_S). No hash-locked condition is installed at this stage.
- 5: **Off-chain negotiation (loop):**
 Π .Update: agree on batches $S_1, S_2, \dots, S_k \subseteq \{a_1, \dots, a_m\}$. Each S_j is embedded in a signed off-chain state σ_j ; the aggregated state σ encodes the intended settlement set $\bigcup_j S_j$ and the net payment.
- 6: **On-chain locking (commit final state):**
 Π .Lock: submit σ to ξ^1 and ξ^2 and install hash-locked conditions $h(\rho)$ with timeouts $T_1 > T_2$ to ensure atomicity.
- 7: **Settlement (on-chain):**
- 8: **if** $\mathcal{B}$ reveals ρ before T_2 **then**
- 9: Π .Unlock on ξ^2 : deliver $\bigcup_j S_j$ to $\mathcal{B}$.
- 10: Π .Unlock on ξ^1 : release the aggregated payment to $\mathcal{S}$ before T_1 .
- 11: **else**
- 12: Π .Refund on ξ^2 : return $\{a_1, \dots, a_m\}$ to $\mathcal{S}$ at T_2 .
- 13: Π .Refund on ξ^1 : return d_B (and any unsettled funds) to $\mathcal{B}$ at T_1 .
- 14: **end if**
- 15: **Channel continuation / closure:** The channel may remain open for further Π .Update rounds without reinstalling locks. Optionally, Π .Close releases remaining deposits and terminates the channel.

Through this process, multiple RWAs can be traded within the channel via successive Update operations, while only a single round of on-chain Unlock or Refund is needed to settle the results. This reduces on-chain congestion and lowers the overall settlement costs. The detailed procedure of the cross-chain channel for multiple RWAs is summarized in Algorithm 2.

5. Security analysis

In this section, we provide the security analysis of xRWA.

Theorem 1 (Security of xRWA). *Assume that the underlying DID and VC infrastructure is secure, meaning that no PPT adversary can cause a verifier to accept an identity or credential that was not honestly issued by an authorized issuer, except with negligible probability. Assume further that the SPV proof system is sound for transaction inclusion, and that the cross-chain channel relies on a collision-resistant hash function h together with correctly enforced smart contracts operating over immutable and consistent blockchains. Under these assumptions, the xRWA protocol Π_{xRWA} achieves cross-chain authentication correctness and cross-chain settlement atomicity, and therefore satisfies RWA security as defined in Definition 6.*

Proof. Suppose there exists a PPT adversary $\mathcal{A}$ that breaks RWA security of Π_{xRWA} with non-negligible probability. By Definition 6, this means that $\mathcal{A}$ either violates cross-chain authentication correctness or violates cross-chain settlement atomicity with non-negligible probability. We treat these two cases in turn and show that each yields an adversary that contradicts one of the assumed secure components.

First, consider cross-chain authentication correctness. Assume that the event $\text{AuthAcc}_{C_t}(a) \wedge \neg \text{AuthRec}_{C_s}(a)$ occurs with non-negligible probability. Here $\text{AuthAcc}_{C_t}(a)$ denotes that C_t accepts a via a cross-chain authentication claim from C_s , and $\text{AuthRec}_{C_s}(a)$ denotes that C_s contains a valid and confirmed authentication record for a . In xRWA, C_t accepts such a claim only after verifying an SPV proof for a transaction on C_s and checking that the embedded commitment matches a valid credential and issuer information.

We construct a simulator that uses $\mathcal{A}$ to break one of the assumed secure components. The simulator emulates honest parties and the blockchains for $\mathcal{A}$, and forwards its queries to the DID and VC system, or the SPV proof system, depending on the component being tested. Whenever $\mathcal{A}$ causes $\text{AuthAcc}_{C_t}(a) \wedge \neg \text{AuthRec}_{C_s}(a)$, the simulator inspects the transcript. If the SPV verifier accepts a proof for a transaction not contained in any valid block header of C_s , the simulator outputs an attack on SPV soundness. If the transaction exists but its commitment does not correspond to any honestly issued credential, or corresponds to a revoked or nonexistent credential, the simulator outputs an attack on the DID and VC infrastructure. If the credential was honest at issuance but later removed or altered while still being accepted by C_t , the simulator outputs an attack on ledger immutability. Hence, a successful violation of cross-chain authentication correctness by $\mathcal{A}$ leads to a successful attack on one of the assumed secure components with non-negligible probability, contradicting the assumptions. Therefore, the probability that $\mathcal{A}$ causes $\text{AuthAcc}_{C_t}(a) \wedge \neg \text{AuthRec}_{C_s}(a)$ must be negligible.

Next, consider cross-chain settlement atomicity. Assume that the event $\text{Com}_1 \oplus \text{Com}_2$ occurs with non-negligible probability, where Com_i denotes that chain C_i applies the settlement updates and $\oplus$ denotes exclusive or. In xRWA, all settlements are executed through the underlying cross-chain channel, which relies on the collision-resistant hash function h , correctly enforced smart contracts, and immutable blockchains, and which by assumption ensures atomic settlement.

We construct a simulator that uses $\mathcal{A}$ to attack the cross-chain channel. The simulator runs $\mathcal{A}$ internally, forwards its messages to the channel challenger, and uses the challenger to emulate locking, settlement, and refund behavior. If $\mathcal{A}$ produces an execution in which exactly one chain commits the settlement, the simulator outputs this as an attack on channel atomicity, contradicting the assumed security of the channel. Consequently, the probability that $\mathcal{A}$ causes $\text{Com}_1 \oplus \text{Com}_2$ must be negligible.

Combining the two parts, any PPT adversary that breaks RWA security of Π_{xRWA} would enable an attack on one of the assumed secure components with non-negligible probability, contradicting their security. Hence, Π_{xRWA} satisfies cross-chain authentication correctness and cross-chain settlement atomicity, and therefore satisfies RWA security. $\square$

6. Implementation and performance evaluation

In this section, we present our concrete implementation of xRWA and test its performance.

6.1. Implementation

The experiments are conducted on a workstation with an Intel Core i9-13980HX CPU and 32 GB RAM, running Windows 11 (24H2). Python 3.12.10 is used to implement and benchmark the VC routines and the SPV algorithm. The reported execution times are obtained from these Python implementations under multi-threaded settings. Smart-contract experiments are performed on Remix VM (Prague) using Solidity 0.8.30 to measure gas consumption for the HTLC and cross-chain channel workflows. The

Table 1
The size of VC across different RWA types.

Type	Size (KB)
Vehicle	7.11
RE	8.05
Gold	6.95
Art	7.38
Bond	7.09
Fund	7.27
IP	7.10
Average	7.27

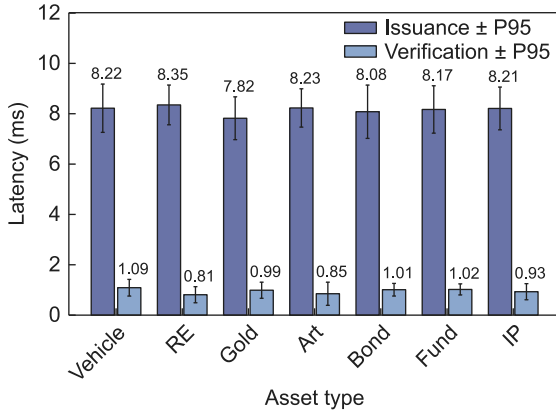


Fig. 3. Latency in issuing and verifying VCs.

entire codebase consists of more than 2000 lines, including the Python implementations for VC/SPV and the Solidity contracts for on-chain evaluation. Our code repository is publicly available at <https://github.com/ZhuHaoming2005/xRWA>.

6.2. Performance evaluation

Table 1 presents the sizes (in KB) of VCs across different asset types. The covered categories include vehicles, real estate (RE), gold, art, bonds, funds, and intellectual property (IP). The sizes range from 6.95 KB to 8.05 KB, with an overall average of 7.27 KB.

Fig. 3 reports the performance of issuance and verification under a multi-threaded workload (500 credentials, 100 iterations, 8 worker threads). The average issuance latency is 8.16 ms per VC, with a P95 latency of 9.04 ms. Verification is faster, with an average latency of 0.96 ms per VC and a P95 of 1.27 ms. These results show that credential issuance requires on the order of several milliseconds per VC, while verification can be performed in less than one millisecond on average. The measured latencies indicate that both operations are lightweight and can be executed efficiently under multi-threaded workloads.

Fig. 4 reports the empirical validation of the $\mathcal{O}(\log n)$ complexity of SPV verification. The measured latency follows the fitted logarithmic model, where n denotes the number of transactions per block.

$$t_{\text{SPV}}(n) \approx 0.69 \times \log_2(n) + 0.23 \mu\text{s}.$$

The experiments cover block sizes from 2^5 to 2^{13} (32 to 8192 transactions). Verification time increases from approximately 3.75 μs at $n = 32$ to 9.27 μs at $n = 8192$, which is consistent with the expected logarithmic trend. This scalability shows that SPV can provide efficient proof validation even for large block sizes, making it a suitable basis for cross-chain RWA authentication.

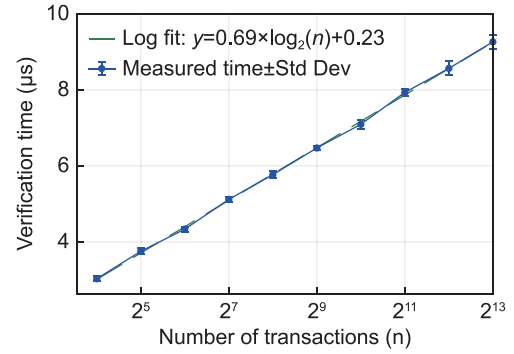


Fig. 4. Verification time of SPV proofs under varying transaction counts.

For the cross-chain channel, handling n cross-chain interactions requires only 917,253 gas in total, since all updates are performed off-chain and only a small constant number of operations are executed on-chain. In contrast, the HTLC scheme consumes $465,426 \times n$ gas for n interactions, as each interaction must execute both the lock and unlock operations on-chain, leading to a linear increase in cost. Moreover, compared with designs that close the channel after each cross-chain settlement, our workflow allows the channel to remain open so that it can either support the next cross-chain interaction or function as an intermediate relay, similar to a payment channel network, thereby avoiding the additional costs caused by repeatedly opening and closing the channel.

In addition to the cost advantage, the channel mechanism also improves efficiency. The time overhead of cross-chain protocols is dominated by blockchain confirmation delays. Processing n interactions with HTLC requires $4 \times n$ on-chain operations across two blockchains, and each operation incurs confirmation latency. By contrast, the cross-chain channel performs only four on-chain operations in total, regardless of n . Therefore, the channel mechanism provides both lower on-chain cost and higher efficiency compared with HTLC, particularly as the number of interactions increases.

7. Conclusion

This paper presents a cross-chain framework for RWAs that addresses identification, authentication, and interaction in a unified workflow. The design leverages DIDs and VCs for decentralization, applies an SPV-based protocol to prevent redundant verification, and introduces a cross-chain channel that enables partial settlement without closure. Experiments indicate that the framework reduces on-chain costs and improves efficiency in cross-chain settings. As future work, we explore potential security vulnerabilities in RWA projects and develop mitigation strategies to strengthen their reliability in practice.

CRedit authorship contribution statement

Yihao Guo: Writing – review & editing, Writing – original draft. **Haoming Zhu:** Writing – original draft. **Minghui Xu:** Supervision, Funding acquisition, Software, Writing – review & editing. **Xiuzhen Cheng:** Writing – review & editing. **Bin Xiao:** Supervision, Funding acquisition, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported in part by NSFC/RGC Joint Research Scheme (N_PolyU529/22, 62261160391), the National Natural Science Foundation of China (62232010, 62302266, U23A20302, U24A20244), and the Shandong Science Fund for Excellent Young Scholars (2023HWYQ-008).

References

- [1] S. Chen, M. Jiang, X. Luo, Exploring the security issues of real world assets (RWA), in: *Proceedings of the Workshop on Decentralized Finance and Security*, 2024, pp. 31–40.
- [2] CZIFI, Overview of the 2025 global stablecoin market, 2025, <https://news.qq.com/rain/a/20250707A07OWI00>. (Accessed 29 August 2025).
- [3] M. Xu, Y. Guo, C. Liu, Q. Hu, D. Yu, Z. Xiong, D.T. Niyato, X. Cheng, Exploring blockchain technology through a modular lens: A survey, *ACM Comput. Surv.* 56 (9) (2024).
- [4] Y. Guo, Z. Wan, X. Cheng, When blockchain meets smart grids: A comprehensive survey, *High-Confidence Comput.* 2 (2) (2022) 100059.
- [5] K. Gai, Y. Wu, L. Zhu, K.-K.R. Choo, B. Xiao, Blockchain-enabled trustworthy group communications in UAV networks, *IEEE Trans. Intell. Transp. Syst.* 22 (7) (2020) 4118–4130.
- [6] CoinMarketCap, Today's cryptocurrency prices by market cap, 2024, <https://coinmarketcap.com>. (Accessed 30 May 2025).
- [7] L. Lao, Z. Li, S. Hou, B. Xiao, S. Guo, Y. Yang, A survey of IoT applications in blockchain systems: Architecture, consensus, and traffic modeling, *ACM Comput. Surv.* 53 (1) (2020) 1–32.
- [8] R. Song, B. Xiao, Y. Song, S. Guo, Y. Yang, A survey of blockchain-based schemes for data sharing and exchange, *IEEE Trans. Big Data* 9 (6) (2023) 1477–1495.
- [9] R. Belchior, A. Vasconcelos, S. Guerreiro, M. Correia, A survey on blockchain interoperability: Past, present, and future trends, *ACM Comput. Surv.* 54 (8) (2021) 1–41.
- [10] Y. Guo, M. Xu, D. Yu, Y. Yu, R. Ranjan, X. Cheng, Cross-channel: Scalable off-chain channels supporting fair and atomic cross-chain operations, *IEEE Trans. Comput.* (2023).
- [11] Y. Guo, M. Xu, X. Cheng, D. Yu, W. Qiu, G. Qu, W. Wang, M. Song, Zkcross: A novel architecture for cross-chain privacy-preserving auditing, in: *33th USENIX Security Symposium, USENIX Security 24*, 2024.
- [12] V. Martin, Comparing number of transactions, 2025, <https://coincodex.com/article/63721/comparing-tps-bitcoin-visa-paypal-and-more>. (Accessed 29 August 2025).
- [13] X. Jia, Z. Yu, J. Shao, R. Lu, G. Wei, Z. Liu, Cross-chain virtual payment channels, *IEEE Trans. Inf. Forensics Secur.* (2023).
- [14] O. Fernau, Makerdao may hook up with traditional bank, 2023, <https://thedefiant.io/news/defi/makerdao-bank-deal-dai-loans>. (Accessed 30 August 2025).
- [15] I. Allison, Hsbc's gold token goes live for retail investors in Hong Kong, 2024, <https://www.coindesk.com/policy/2024/03/27/hsbcs-gold-token-goes-live-for-retail-investors-in-hong-kong>. (Accessed 2 september 2025).
- [16] Securitize, The leader in real-world asset tokenization, 2025, <https://securitize.io/>. (Accessed 29 August 2025).
- [17] Brickken, Tokenize, trade, and manage any financial instrument, 2025, <https://www.brickken.com/en>. (Accessed 29 August 2025).
- [18] BNY, BNY and goldman sachs launch tokenized money market funds solution, 2025, <https://www.goldmansachs.com/pressroom/press-releases/2025/bny-goldman-sachs-launch-tokenized-money-market-funds-solution>. (Accessed 29 August 2025).
- [19] Finance, Real-world asset tokenization market has grown almost fivefold in 3 years, 2025, <https://www.coindesk.com/business/2025/06/26/real-world-asset-tokenization-market-has-grown-almost-fivefold-in-3-years>. (Accessed 20 November 2025).
- [20] S. Koc, Escrow mechanisms for tokenization of real world assets (RWA) digital trust for physical wealth, *Authorea Prepr.* (2024).
- [21] X. Zhao, J. Ding, Y. Su, H. Wang, F. Guo, Q. Zhang, M. Mu, Scalable & secure real-world asset tokenization using ethereum staking & layer-2 solutions, *Peer-To-Peer Netw. Appl.* 18 (5) (2025) 1–32.
- [22] N. Xia, X. Zhao, Y. Yang, Y. Li, Y. Li, Exploration on real world assets and tokenization, 2025, arXiv preprint [arXiv:2503.01111](https://arxiv.org/abs/2503.01111).
- [23] S. Ling, Y. Du, Y. Zhou, L. Wu, C. Wang, X. Jia, H. Yan, Sok: Stablecoin designs, risks, and the stablecoin LEGO, 2025, arXiv preprint [arXiv:2506.17622](https://arxiv.org/abs/2506.17622).
- [24] W3C, Decentralized identifiers (DIDs) v1.0, 2022, <https://www.w3.org/TR/did-core/>. (Accessed 29 August 2025).
- [25] C. Mazzocca, A. Acar, S. Uluagac, R. Montanari, P. Bellavista, M. Conti, A survey on decentralized identifiers and verifiable credentials, *IEEE Commun. Surv. Tutor.* (2025).
- [26] W3C, Verifiable credentials data model v2.0, 2025, <https://www.w3.org/TR/vc-data-model-2.0/>. (Accessed 29 August 2025).
- [27] T. Xie, K. Gai, J. Yu, L. Zhu, B. Xiao, SLVC-DIDA: Signature-less verifiable credential-based issuer-hiding and multi-party authentication for decentralized identity, 2025, arXiv preprint [arXiv:2501.11052](https://arxiv.org/abs/2501.11052).
- [28] Z. Ge, J. Gu, C. Wang, Y. Long, X. Xu, D. Gu, Accio: Variable-amount, optimized-unlinkable and nizk-free off-chain payments via hubs, in: *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, 2023, pp. 1541–1555.